

Spring Boot Practice Project

Your Tasks: Finish the Other Classes

In this project, the **Person** part is already finished. It is your example. Please copy the same way of working to finish three more things: **Task**, **PhoneNumber**, and **UserName**.

For trainees learning Spring Boot for the first time
Project folder: **com.cl.demo**

1. What is this project?

This is a small Spring Boot project. It keeps data in memory (in simple lists), not in a real database. This makes it easy to learn.

The **Person** part is already done from start to end. You can open it and see how it works. Some other things are only half done. Your job is to finish them by copying the same style as Person.

A few words you will see (in easy language)

Word	What it means in this project
Entity	A simple class that holds data (for example: a Person has a name and email).
Request object	A class that holds the data the user sends to us.
Response object	A class that holds the data we send back to the user.
Service	A class that does the real work (add, find, update, delete).
Controller	A class that receives web requests and calls the Service.
List	A place in memory where we keep all the records.

What is finished and what is not

Thing	Entity	Request/Response	Service	Controller
Person	Done	Done	Done	Done
Task	Done	Not done	Not done	Empty class only
PhoneNumber	Done	Not done	Not done	Not done
UserName	Done	Not done	Used inside Person	Not done

Note: 'Empty class only' means the file exists but has no code inside yet. You will write the code.

2. Look at Person first (your example)

Before you write anything, open the Person files and read them slowly. Every 'thing' in this project is built from the same five classes. Person has all five:

Class	Folder	Person example	Job
Entity	entities	Person.java	Holds the data
Create request	requestobjects	PersonCreateRequest	Data the user sends to create
Update request	requestobjects	PersonUpdateRequest	Id + data the user wants to change
Create response	responseobjects	PersonCreateResponse	Data we send back after create/read
Update response	responseobjects	PersonUpdateResponse	Data we send back after update
Service	services	PersonService	Does the work on the list
Controller	controllers	PersonController	Handles the web address

Rules that Person follows (please do the same)

- Every Entity uses BaseClass. So it already has id, isActive, createdAt, and updatedAt. Do not write these again.
- When you **create** a record: give it a new id with `UUID.randomUUID()`, set `isActive` to true, set `createdAt` to `new Date()`, then add it to the list.
- When you **read** records: skip any record where `isActive` is false.
- When you **delete**: do not really remove the record. Just set `isActive` to false. So it becomes hidden. This is safer.
- When you **update**: use `HelperUtils.compare(old, new)` to choose the value.
- The Controller should be short. It only calls the Service and then uses the `convert(...)` method to make the response. Do not send the Entity directly.

Best tip: The fastest way to finish a task is: open the matching Person file, copy it, and change the names. For example, copy `PersonService` to make `TaskService`.

3. Task 1 — Finish “Task”

DO THIS FIRST

The Task entity is ready. The list Task_List is also ready. But TaskController is an empty class, and there is no TaskService. Please build them like Person.

A Task has these fields: title, description, taskNumber, dueDate, startDate, endDate, taskStatus, isAssigned (and the fields from BaseClass).

Step 1 — Make the request classes

- **TaskCreateRequest** (in the requestobjects folder). Use @Getter @Setter on top. Fields: title, description, dueDate, startDate, taskStatus, isAssigned. Do not add taskNumber here — the program will make it for you.
- **TaskUpdateRequest**. Fields: uuid and the fields the user can change, for example titleToUpdate, descriptionToUpdate, taskStatusToUpdate, dueDateToUpdate, isAssignedToUpdate.

Step 2 — Make the response classes

- **TaskCreateResponse** and **TaskUpdateResponse** (in the responseobjects folder). Copy the two convert methods from PersonCreateResponse (one for a single Task, one for a list of Tasks).
- Show these fields to the user: taskId, title, description, taskNumber, taskStatus, dueDate, isAssigned. Change dates and status into text (String).

Step 3 — Make TaskService

Put @Service on top. Write these methods:

Method	Gives back	What it does
addTask(...)	Map (text)	Make a Task, set id/isActive/createdDate, make a taskNumber, add to list
getTaskById(uuid)	Task	Find the active Task. If not found, return an empty Task
getAllTasks()	List of Task	Return all Tasks where isActive is true
updateTask(...)	Task	Use compare() for each field, set updatedDate, put back in list
deleteById(uuid)	Boolean	Set isActive to false (hide it)
generateTaskNumber()	String	Make a code, for example "TASK-" + a short number

Here is a small start for you:

```
@Service
public class TaskService {
    public static final String TASK_SAVED = "Task saved";

    public Map<String,String> addTask(TaskCreateRequest req) {
        Map<String,String> response = new HashMap<>();
        Task task = new Task();
        task.setId(UUID.randomUUID());
        task.setIsActive(Boolean.TRUE);
        task.setCreatedDate(new Date());
        task.setTaskNumber(generateTaskNumber());
        // now set title, description, dates from req
        if (DemoApplication.Task_List.add(task)) {
            response.put("response", TASK_SAVED);
        }
    }
}
```

```
        }  
        return response;  
    }  
}
```

Step 4 — Fill in TaskController

Open the empty TaskController. Add `@RestController` and `@RequestMapping("task")`. Bring in TaskService with `@Autowired`. Then make these web addresses:

Type	Web address	What it does
POST	/task/add	Call addTask
GET	/task/getById?uuid=	Call getTaskById, return TaskCreateResponse
GET	/task/getAll	Call getAllTasks, return a list
PUT	/task/update	Call updateTask, return TaskUpdateResponse
DELETE	/task/deleteById?id=	Call deleteById, return true or false

4. Task 2 — Finish “PhoneNumber”

DO THIS SECOND

The PhoneNumber entity is ready. It has countryCode and phoneNumber. Inside PersonService there is a line //TODO: Add Phone Number Logic. You will build PhoneNumber and connect it to Person.

Step 1 — Make a list to keep phone numbers

Open DemoApplication and add one new line, the same way as the other lists:

```
public static List<PhoneNumber> PhoneNumber_List = new ArrayList<>();
```

Step 2 — Make the request and response classes

- **PhoneNumberCreateRequest**: countryCode (String) and phoneNumber (Long).
- **PhoneNumberUpdateRequest**: uuid, countryCodeToUpdate, phoneNumberToUpdate.
- **PhoneNumberCreateResponse** and **PhoneNumberUpdateResponse**: copy the two convert methods from Person. Show phoneNumberId, countryCode, phoneNumber.

Step 3 — Make PhoneNumberService

Method	Gives back	What it does
addPhoneNumber(...)	PhoneNumber	Make it, set id/isActive/createdDate, add to list, return it
getPhoneNumberById(uuid)	PhoneNumber	Find the active one, or return an empty one
getAllPhoneNumbers()	List	Return all where isActive is true
updatePhoneNumber(...)	PhoneNumber	Use compare(), set updatedDate
deleteById(uuid)	Boolean	Set isActive to false

Step 4 — Make PhoneNumberController

Use @RestController and @RequestMapping("phoneNumber"):

Type	Web address	What it does
POST	/phoneNumber/add	Call addPhoneNumber
GET	/phoneNumber/getById?uuid=	Call getPhoneNumberById
GET	/phoneNumber/getAll	Call getAllPhoneNumbers
PUT	/phoneNumber/update	Call updatePhoneNumber
DELETE	/phoneNumber/deleteById?id=	Call deleteById

Step 5 — Connect it to Person

- In PersonService, bring in PhoneNumberService with @Autowired.
- Find the //TODO line in addPerson. Remove it. Make a PhoneNumber from the request and call person.setPhoneNumber(...). For this, add two fields to PersonCreateRequest: personCountryCode and personPhoneNumber.
- In PersonCreateResponse.convert there is one line about phone number that is turned off (commented). Turn it on, but first check the phone number is not null, then show it as countryCode + " " + phoneNumber.

5. Task 3 — Clean up “UserName”

DO THIS LAST

UserName belongs to Person. It keeps the active name and a list of old names. It does not need its own Controller. Here you only make the code cleaner and fix a small mistake.

- Look at `getUserNameByCompare` in `PersonService`. There is a small mistake: when it sets the new active name, it always uses the value from the request, even when `compare()` chose the old value. Please set the active name to the value that `compare()` returned.
- In the response convert methods, check that `getUserName()` is not null before you call `getActiveUserName()`. If you do not check, the program can crash.
- You can also add one read address in `PersonController` to show the list of a person's old user names. (This part is optional.)

6. Small mistakes to fix (in all parts)

While you work, please fix these small problems. They are already in the Person code, so fix them everywhere you copy from Person.

- 1 Empty result problem.** `getPersonById` returns new `Person()` when nothing is found. Then convert tries `getId().toString()` and the program crashes. Please check for null or empty before using it.
- 2 Check before you save.** `verifyUserNameAndEmail` adds the email and username into the sets while it is only checking. So a failed create still fills the sets. Please only add them after the create is successful.
- 3 Remove the extra import.** In `PersonService`, the line `import org.apache.catalina.User;` is not used. Delete it.
- 4 Boolean check.** Change `getIsActive() != false` and `== true` to a simple boolean check, for example just `getIsActive()`.
- 5 Update date.** Every update method should set `updatedAt = new Date()`. Person's update does not do this now.
- 6 Empty fields.** Before you save, check that important fields (title, email, phone number) are not empty.

7. Your finish list (tick each box)

For Task and for PhoneNumber, check every line:

No.	What to check	Done?
1	Create request and Update request classes are made (with @Getter @Setter)	
2	Create response and Update response classes have both convert() methods	
3	Service class has @Service on top	
4	add / getById / getAll / update / deleteById methods are written	
5	Delete only sets isActive to false; reading skips hidden records	
6	Controller has @RestController and @RequestMapping	
7	All 5 web addresses work and return response objects (not the entity)	
8	PhoneNumber is connected to Person; the //TODO line is removed	
9	The small mistakes in Part 6 are fixed	

No.	What to check	Done?
10	You tested each address (good case and missing-id case)	

Remember: Use the same names as Person. For Task, use TaskCreateRequest, TaskService, @RequestMapping("task"), and the same method names (add, getByld, getAll, update, deleteByld). Same names make the code easy to read.